

armBot Documentation

[File: rosToPdf.py \(Other\)](#)

```
1: import os
2: from reportlab.lib.pagesizes import letter
3: from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Preformatted, PageBreak
4: from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
5: from reportlab.lib import colors
6:
7: def get_ros_file_type(filename, path):
8:     """Determine the type of ROS file based on location and extension."""
9:     if 'launch' in path:
10:         return 'Launch File'
11:     elif 'config' in path:
12:         return 'Configuration File'
13:     elif 'srv' in path:
14:         return 'Service Definition'
15:     elif 'action' in path:
16:         return 'Action Definition'
17:     elif 'urdf' in path:
18:         return 'URDF File'
19:     elif path.endswith('/src'):
20:         return 'Source Code'
21:     elif 'include' in path:
22:         return 'Header File'
23:     else:
24:         return 'Other'
25:
26: def is_relevant_file(filename):
27:     """Check if the file should be included in documentation."""
28:     relevant_extensions = {
29:         '.py', '.cpp', '.hpp', '.h', '.xml', '.launch', '.yaml',
30:         '.urdf', '.xacro', '.srv', '.action', '.msg', '.ino'
31:     }
32:     return any(filename.lower().endswith(ext) for ext in relevant_extensions)
33:
34: def create_section_heading(text, level=1):
35:     """Create section heading with different styles based on level."""
36:     styles = getSampleStyleSheet()
37:     if level == 1:
38:         style = ParagraphStyle(
39:             'Heading1',
40:             parent=styles['Heading1'],
41:             fontSize=16,
42:             spaceAfter=20,
43:         )
44:     else:
45:         style = ParagraphStyle(
46:             'Heading2',
```

```

47:         parent=styles['Heading2'],
48:         fontSize=14,
49:         spaceAfter=15,
50:         leftIndent=20,
51:     )
52:     return Paragraph(text, style)
53:
54: def process_directory(root_path, base_path):
55:     """Recursively process directory and return content structure."""
56:     content_structure = []
57:
58:     for dirpath, dirnames, filenames in os.walk(root_path):
59:         # Skip hidden directories and files
60:         dirnames[:] = [d for d in dirnames if not d.startswith('.')]
61:         filenames = [f for f in filenames if not f.startswith('.') and is_relevant_file(f)]
62:
63:         if not filenames:
64:             continue
65:
66:         # Get relative path for section heading
67:         rel_path = os.path.relpath(dirpath, base_path)
68:         if rel_path != '.':
69:             content_structure.append({
70:                 'type': 'section',
71:                 'text': rel_path,
72:                 'level': len(rel_path.split(os.sep))
73:             })
74:
75:         for filename in sorted(filenames):
76:             file_path = os.path.join(dirpath, filename)
77:             try:
78:                 with open(file_path, 'r', encoding='utf-8') as file:
79:                     content = file.read()
80:                     file_type = get_ros_file_type(filename, dirpath)
81:                     content_structure.append({
82:                         'type': 'file',
83:                         'name': filename,
84:                         'content': content,
85:                         'file_type': file_type
86:                     })
87:             except Exception as e:
88:                 print(f"Error reading {file_path}: {str(e)}")
89:
90:     return content_structure
91:
92: def create_pdf(project_path, output_pdf):
93:     """Create PDF documentation from ROS project structure."""
94:     doc = SimpleDocTemplate(output_pdf, pagesize=letter)
95:     styles = getSampleStyleSheet()
96:
97:     # Custom styles
98:     code_style = ParagraphStyle(
99:         'CodeStyle',

```

```

100:         parent=styles['Normal'],
101:         fontName='Courier',
102:         fontSize=9,
103:         backgroundColor=colors.lightgrey,
104:         leftIndent=20,
105:     )
106:
107:     file_info_style = ParagraphStyle(
108:         'FileInfo',
109:         parent=styles['Normal'],
110:         fontSize=10,
111:         textColor=colors.blue,
112:         leftIndent=20,
113:     )
114:
115:     content = []
116:
117:     # Add title
118:     project_name = os.path.basename(project_path)
119:     content.append(Paragraph(f"{project_name} Documentation", styles['Title']))
120:     content.append(Spacer(1, 30))
121:
122:     # Process the project structure
123:     structure = process_directory(project_path, project_path)
124:
125:     for item in structure:
126:         if item['type'] == 'section':
127:             content.append(create_section_heading(item['text'], item['level']))
128:         else:
129:             # Add file information
130:             content.append(Paragraph(
131:                 f"File: {item['name']} ({item['file_type']})",
132:                 file_info_style
133:             ))
134:             content.append(Spacer(1, 6))
135:
136:             # Add file content
137:             numbered_lines = [f"{i+1}: {line}" for i, line in enumerate(item['content'].splitlines())]
138:             formatted_content = '\n'.join(numbered_lines)
139:             content.append(Preformatted(formatted_content, code_style))
140:             content.append(Spacer(1, 20))
141:
142:             # Add page break after each file for better readability
143:             content.append(PageBreak())
144:
145:     # Build PDF
146:     doc.build(content)
147:
148: if __name__ == '__main__':
149:     # Example usage
150:     project_path = 'C:/Users/jewel.nath/Desktop/armBot' # Replace with your armBot folder
151:     output_pdf = 'armbot_code.pdf'
152:     create_pdf(project_path, output_pdf)

```


src\armbot_bringup

File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.ros.org/schema/package_format3"?>
3: <package format="3">
4:   <name>armbot_bringup</name>
5:   <version>0.0.0</version>
6:   <description>Armbot Bringup package</description>
7:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
8:   <license>Apache 2.0</license>
9:
10:   <buildtool_depend>ament_cmake</buildtool_depend>
11:
12:   <exec_depend>ros2launch</exec_depend>
13:   <exec_depend>armbot_controller</exec_depend>
14:   <exec_depend>armbot_description</exec_depend>
15:   <exec_depend>armbot_moveit</exec_depend>
16:   <exec_depend>armbot_remote</exec_depend>
17:
18:   <test_depend>ament_lint_auto</test_depend>
19:   <test_depend>ament_lint_common</test_depend>
20:
21:   <export>
22:     <build_type>ament_cmake</build_type>
23:   </export>
24: </package>
```

src\armbot_bringup\launch

File: real_robot.launch.py (Launch File)

```
1: import os
2: from launch import LaunchDescription
3: from launch.actions import IncludeLaunchDescription
4: from ament_index_python.packages import get_package_share_directory
5:
6:
7: def generate_launch_description():
8:     controller = IncludeLaunchDescription(
9:         os.path.join(
10:             get_package_share_directory("armbot_controller"),
11:             "launch",
12:             "controller.launch.py"
13:         ),
14:         launch_arguments={"is_sim": "False"}.items()
15:     )
16:
17:     moveit = IncludeLaunchDescription(
18:         os.path.join(
19:             get_package_share_directory("armbot_moveit"),
20:             "launch",
21:             "moveit.launch.py"
22:         ),
23:         launch_arguments={"is_sim": "False"}.items()
24:     )
25:
26:     remote_interface = IncludeLaunchDescription(
27:         os.path.join(
28:             get_package_share_directory("armbot_remote"),
29:             "launch",
30:             "remote_interface.launch.py"
31:         ),
32:     )
33:
34:     return LaunchDescription([
35:         controller,
36:         moveit,
37:         remote_interface,
38:     ])
```

[File: simulated_robot.launch.py \(Launch File\)](#)

```
1: import os
2: from launch import LaunchDescription
3: from launch.actions import IncludeLaunchDescription
4: from ament_index_python.packages import get_package_share_directory
5:
6:
7: def generate_launch_description():
8:     gazebo = IncludeLaunchDescription(
9:         os.path.join(
10:             get_package_share_directory("armbot_description"),
11:             "launch",
12:             "gazebo.launch.py"
13:         )
14:     )
15:
16:     controller = IncludeLaunchDescription(
17:         os.path.join(
18:             get_package_share_directory("armbot_controller"),
19:             "launch",
20:             "controller.launch.py"
21:         ),
22:         launch_arguments={"is_sim": "True"}.items()
23:     )
24:
25:     moveit = IncludeLaunchDescription(
26:         os.path.join(
27:             get_package_share_directory("armbot_moveit"),
28:             "launch",
29:             "moveit.launch.py"
30:         ),
31:         launch_arguments={"is_sim": "True"}.items()
32:     )
33:
34:
35:     return LaunchDescription([
36:         gazebo,
37:         controller,
38:         moveit
39:     ])
```

src\armbot_controller

[File: armbot_controller.xml \(Other\)](#)

```
1: <library path="armbot_controller">
2:   <class name="armbot_controller/ArmbotInterface"
3:     type="armbot_controller::ArmbotInterface"
4:     base_class_type="hardware_interface::SystemInterface"
5:   >
6:     <description>
7:       Armbot Hardware Interface
8:     </description>
9:   </class>
10: </library>
```


File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <package format="3">
3:   <name>armbot_controller</name>
4:   <version>0.0.0</version>
5:   <description>The armbot_controller package</description>
6:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
7:   <author email="jewelnath.me@gmail.com">Jewel Nath</author>
8:   <license>Apache 2.0</license>
9:
10:   <!-- Build dependencies -->
11:   <buildtool_depend>ament_cmake</buildtool_depend>
12:   <buildtool_depend>ament_cmake_python</buildtool_depend>
13:
14:   <!-- Runtime dependencies -->
15:   <depend>rclcpp</depend>
16:   <depend>rclpy</depend>
17:   <depend>trajectory_msgs</depend>
18:   <depend>sensor_msgs</depend>
19:   <depend>hardware_interface</depend>
20:   <depend>rclcpp_lifecycle</depend>
21:   <depend>pluginlib</depend>
22:   <depend>libserial-dev</depend>
23:
24:   <!-- Execution dependencies -->
25:   <exec_depend>ros2launch</exec_depend>
26:   <exec_depend>robot_state_publisher</exec_depend>
27:   <exec_depend>xacro</exec_depend>
28:   <exec_depend>controller_manager</exec_depend>
29:   <exec_depend>armbot_description</exec_depend>
30:   <exec_depend>joint_state_publisher_gui</exec_depend>
31:
32:   <!-- Testing dependencies -->
33:   <test_depend>ament_lint_auto</test_depend>
34:   <test_depend>ament_lint_common</test_depend>
35:
36:   <export>
37:     <build_type>ament_cmake</build_type>
38:   </export>
39: </package>
40:
```

src\armbot_controller\armbot_controller

File: [__init__.py](#) (Other)

1:

File: slider_control.py (Other)

```
1: #!/usr/bin/env python3
2: import rclpy
3: from rclpy.node import Node
4: from sensor_msgs.msg import JointState
5: from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
6:
7:
8: class SliderControl(Node):
9:
10:     def __init__(self):
11:         super().__init__("slider_control")
12:         self.arm_pub_ = self.create_publisher(JointTrajectory, "arm_controller/joint_trajectory", 10)
13:         self.gripper_pub_ = self.create_publisher(JointTrajectory, "gripper_controller/joint_trajectory", 10)
14:         self.sub_ = self.create_subscription(JointState, "joint_commands", self.sliderCallback, 10)
15:         self.get_logger().info("Slider Control Node started")
16:
17:         # Define joint names for the arm and gripper
18:         self.arm_joint_names = ["joint_1", "joint_2", "joint_3", "joint_4", "joint_5"]
19:         self.gripper_joint_names = ["joint_6"]
20:
21:     def sliderCallback(self, msg):
22:         if len(msg.position) < 6:
23:             self.get_logger().warn("Received joint command message with insufficient position")
24:             return
25:
26:         # Prepare JointTrajectory messages for arm and gripper
27:         arm_controller = JointTrajectory()
28:         gripper_controller = JointTrajectory()
29:         arm_controller.joint_names = self.arm_joint_names
30:         gripper_controller.joint_names = self.gripper_joint_names
31:
32:         # Set positions for arm joints
33:         arm_goal = JointTrajectoryPoint()
34:         arm_goal.positions = msg.position[:5] # Assuming first 5 positions are for arm joints
35:         arm_goal.time_from_start.sec = 1 # Example timing
36:
37:         # Set position for gripper joint (assuming it's the 6th joint in msg.position)
38:         gripper_goal = JointTrajectoryPoint()
39:         gripper_goal.positions = [msg.position[5]] # Assuming 6th position is for gripper joint
40:         gripper_goal.time_from_start.sec = 1 # Example timing
41:
42:         # Append the trajectory points to respective controllers
43:         arm_controller.points.append(arm_goal)
44:         gripper_controller.points.append(gripper_goal)
45:
46:         # Publish the JointTrajectory messages
47:         self.arm_pub_.publish(arm_controller)
48:         self.gripper_pub_.publish(gripper_controller)
49:         self.get_logger().info("Published joint trajectory commands.")
50:
51: def main():
```

```
52:     rclpy.init()
53:
54:     simple_publisher = SliderControl()
55:     rclpy.spin(simple_publisher)
56:
57:     simple_publisher.destroy_node()
58:     rclpy.shutdown()
59:
60:
61: if __name__ == '__main__':
62:     main()
63:
```

src\armbot_controller\config

File: [armbot_controllers.yaml](#) (Configuration File)

```
1: controller_manager:
2:   ros__parameters:
3:     update_rate: 10 # Hz
4:
5:   arm_controller:
6:     type: joint_trajectory_controller/JointTrajectoryController
7:
8:   gripper_controller:
9:     type: joint_trajectory_controller/JointTrajectoryController
10:
11:   joint_state_broadcaster:
12:     type: joint_state_broadcaster/JointStateBroadcaster
13:
14: arm_controller:
15:   ros__parameters:
16:     joints:
17:       - joint_1
18:       - joint_2
19:       - joint_3
20:       - joint_4
21:       - joint_5
22:
23:     command_interfaces:
24:       - position
25:
26:     state_interfaces:
27:       - position
28:
29:     open_loop_control: true
30:     allow_integration_in_goal_trajectories: true
31:
32: gripper_controller:
33:   ros__parameters:
34:     joints:
35:       - joint_6
36:
37:     command_interfaces:
38:       - position
39:
40:     state_interfaces:
41:       - position
42:
43:     open_loop_control: true
44:     allow_integration_in_goal_trajectories: true
45:
```

src\armbot_controller\include\armbot_controller

File: armbot_interface.hpp (Header File)

```
1: #ifndef ARMBOT_INTERFACE_H_
2: #define ARMBOT_INTERFACE_H_
3:
4: #include <rclcpp/rclcpp.hpp>
5: #include <hardware_interface/system_interface.hpp>
6: #include <libserial/SerialPort.h>
7: #include <rclcpp_lifecycle/state.hpp>
8: #include <rclcpp_lifecycle/node_interfaces/lifecycle_node_interface.hpp>
9:
10: #include <vector>
11: #include <string>
12:
13: namespace armbot_controller
14: {
15:
16: using CallbackReturn = rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;
17:
18: class ArmbotInterface : public hardware_interface::SystemInterface
19: {
20: public:
21:   ArmbotInterface();
22:   ~ArmbotInterface() override;
23:
24:   // LifecycleNodeInterface methods
25:   CallbackReturn on_activate(const rclcpp_lifecycle::State &previous_state) override;
26:   CallbackReturn on_deactivate(const rclcpp_lifecycle::State &previous_state) override;
27:
28:   // SystemInterface methods
29:   CallbackReturn on_init(const hardware_interface::HardwareInfo &hardware_info) override;
30:   std::vector<hardware_interface::StateInterface> export_state_interfaces() override;
31:   std::vector<hardware_interface::CommandInterface> export_command_interfaces() override;
32:   hardware_interface::return_type read(const rclcpp::Time &time, const rclcpp::Duration &period) override;
33:   hardware_interface::return_type write(const rclcpp::Time &time, const rclcpp::Duration &period) override;
34:
35: private:
36:   LibSerial::SerialPort arduino_;
37:   std::string port_;
38:   std::vector<double> position_commands_;
39:   std::vector<double> prev_position_commands_;
40:   std::vector<double> position_states_;
41:
42:   std::string format_command(const std::string& prefix, int value) const;
43: };
44:
45: } // namespace armbot_controller
46:
47: #endif // ARMBOT_INTERFACE_H_
48:
```


src\armbot_controller\launch

File: [controller.launch.py](#) (Launch File)

```
1: import os
2: from launch import LaunchDescription
3: from launch.actions import DeclareLaunchArgument
4: from launch_ros.actions import Node
5: from launch_ros.parameter_descriptions import ParameterValue
6: from launch.substitutions import Command, LaunchConfiguration
7: from launch.conditions import UnlessCondition
8: from ament_index_python.packages import get_package_share_directory
9:
10: def generate_launch_description():
11:     # Launch configuration for determining if it is a simulation
12:     is_sim = LaunchConfiguration("is_sim")
13:
14:     # Declare the is_sim argument with a default value of "True"
15:     is_sim_arg = DeclareLaunchArgument(
16:         "is_sim",
17:         default_value="True",
18:         description="Set to 'True' if running in simulation mode, 'False' for real hardware
19:     )
20:
21:     # Generate the robot description using Xacro
22:     robot_description = ParameterValue(
23:         Command([
24:             "xacro ",
25:             os.path.join(
26:                 get_package_share_directory("armbot_description"),
27:                 "urdf",
28:                 "armbot.urdf.xacro"
29:             ),
30:             " is_sim:=", is_sim
31:         ]),
32:         value_type=str
33:     )
34:
35:     # Node for publishing the robot state
36:     robot_state_publisher_node = Node(
37:         package="robot_state_publisher",
38:         executable="robot_state_publisher",
39:         parameters=[{"robot_description": robot_description, "use_sim_time": is_sim}],
40:         condition=UnlessCondition(is_sim),
41:     )
42:
43:     # Controller manager node (active only in non-simulation mode)
44:     controller_manager = Node(
45:         package="controller_manager",
46:         executable="ros2_control_node",
47:         parameters=[
48:             {"robot_description": robot_description, "use_sim_time": is_sim},
```



```

49:         os.path.join(
50:             get_package_share_directory("armbot_controller"),
51:             "config",
52:             "armbot_controllers.yaml"
53:         ),
54:     ],
55:     condition=UnlessCondition(is_sim),
56: )
57:
58: # Spawners for controllers
59: joint_state_broadcaster_spawner = Node(
60:     package="controller_manager",
61:     executable="spawner",
62:     arguments=["joint_state_broadcaster", "--controller-manager", "/controller_manager"]
63: )
64:
65: arm_controller_spawner = Node(
66:     package="controller_manager",
67:     executable="spawner",
68:     arguments=["arm_controller", "--controller-manager", "/controller_manager"]
69: )
70:
71: gripper_controller_spawner = Node(
72:     package="controller_manager",
73:     executable="spawner",
74:     arguments=["gripper_controller", "--controller-manager", "/controller_manager"]
75: )
76:
77: # Return a LaunchDescription with all necessary components
78: return LaunchDescription([
79:     is_sim_arg,
80:     robot_state_publisher_node,
81:     controller_manager,
82:     joint_state_broadcaster_spawner,
83:     arm_controller_spawner,
84:     gripper_controller_spawner
85: ])
86:

```

[File: slider_controller.launch.py \(Launch File\)](#)

```
1: import os
2: from launch import LaunchDescription
3: from launch.actions import IncludeLaunchDescription
4: from launch.launch_description_sources import PythonLaunchDescriptionSource
5: from launch_ros.actions import Node
6: from ament_index_python.packages import get_package_share_directory
7:
8: def generate_launch_description():
9:     # Path to the armbot_controller package launch file
10:    controller_launch = IncludeLaunchDescription(
11:        PythonLaunchDescriptionSource(
12:            os.path.join(
13:                get_package_share_directory("armbot_controller"),
14:                "launch",
15:                "controller.launch.py"
16:            )
17:        ),
18:        launch_arguments={"is_sim": "True"}.items()
19:    )
20:
21:    # Node for joint state publisher GUI
22:    joint_state_publisher_gui_node = Node(
23:        package="joint_state_publisher_gui",
24:        executable="joint_state_publisher_gui",
25:        remappings=[
26:            ("/joint_states", "/joint_commands"),
27:        ]
28:    )
29:
30:    # Node for slider control
31:    slider_control_node = Node(
32:        package="armbot_controller",
33:        executable="slider_control"
34:    )
35:
36:    return LaunchDescription([
37:        controller_launch,
38:        joint_state_publisher_gui_node,
39:        slider_control_node
40:    ])
41:
```

src\armbot_controller\src

File: armbot_interface.cpp (Other)

```
1: #include "armbot_controller/armbot_interface.hpp"
2: #include <hardware_interface/types/hardware_interface_type_values.hpp>
3: #include <pluginlib/class_list_macros.hpp>
4: #include <sstream>
5: #include <iomanip>
6:
7: namespace armbot_controller
8: {
9:
10: // Helper function to format a command string with zero-padding
11: std::string ArmbotInterface::format_command(const std::string& prefix, int value) const
12: {
13:     std::ostringstream oss;
14:     oss << prefix << std::setw(3) << std::setfill('0') << value << ",";
15:     return oss.str();
16: }
17:
18: ArmbotInterface::ArmbotInterface() = default;
19: ArmbotInterface::~ArmbotInterface()
20: {
21:     if (arduino_.IsOpen())
22:     {
23:         try
24:         {
25:             arduino_.Close();
26:         }
27:         catch (const std::exception& e)
28:         {
29:             RCLCPP_FATAL_STREAM(rclcpp::get_logger("ArmbotInterface"),
30:                                 "Exception while closing port " << port_ << ": " << e.what());
31:         }
32:     }
33: }
34:
35: CallbackReturn ArmbotInterface::on_init(const hardware_interface::HardwareInfo& hardware_in
36: {
37:     auto result = hardware_interface::SystemInterface::on_init(hardware_info);
38:     if (result != CallbackReturn::SUCCESS)
39:     {
40:         return result;
41:     }
42:
43:     try
44:     {
45:         port_ = info_.hardware_parameters.at("port");
46:     }
47:     catch (const std::out_of_range&)
48:     {
```

```

49:     RCLCPP_FATAL(rclcpp::get_logger("ArmbotInterface"), "No Serial Port provided! Aborting.");
50:     return CallbackReturn::FAILURE;
51: }
52:
53: position_commands_.resize(info_.joints.size(), 0.0);
54: position_states_.resize(info_.joints.size(), 0.0);
55: prev_position_commands_.resize(info_.joints.size(), 0.0);
56:
57: return CallbackReturn::SUCCESS;
58: }
59:
60: std::vector<hardware_interface::StateInterface> ArmbotInterface::export_state_interfaces()
61: {
62:     std::vector<hardware_interface::StateInterface> state_interfaces;
63:     for (size_t i = 0; i < info_.joints.size(); ++i)
64:     {
65:         state_interfaces.emplace_back(hardware_interface::StateInterface(
66:             info_.joints[i].name, hardware_interface::HW_IF_POSITION, &position_states_[i]));
67:     }
68:     return state_interfaces;
69: }
70:
71: std::vector<hardware_interface::CommandInterface> ArmbotInterface::export_command_interfaces()
72: {
73:     std::vector<hardware_interface::CommandInterface> command_interfaces;
74:     for (size_t i = 0; i < info_.joints.size(); ++i)
75:     {
76:         command_interfaces.emplace_back(hardware_interface::CommandInterface(
77:             info_.joints[i].name, hardware_interface::HW_IF_POSITION, &position_commands_[i]));
78:     }
79:     return command_interfaces;
80: }
81:
82: CallbackReturn ArmbotInterface::on_activate(const rclcpp_lifecycle::State&)
83: {
84:     RCLCPP_INFO(rclcpp::get_logger("ArmbotInterface"), "Activating hardware...");
85:
86:     position_commands_.assign(info_.joints.size(), 0.0);
87:     prev_position_commands_.assign(info_.joints.size(), 0.0);
88:     position_states_.assign(info_.joints.size(), 0.0);
89:
90:     try
91:     {
92:         arduino_.Open(port_);
93:         arduino_.SetBaudRate(LibSerial::BaudRate::BAUD_115200);
94:     }
95:     catch (const std::exception& e)
96:     {
97:         RCLCPP_FATAL_STREAM(rclcpp::get_logger("ArmbotInterface"),
98:             "Failed to open port " << port_ << ": " << e.what());
99:         return CallbackReturn::FAILURE;
100:     }
101:

```

```

102:   RCLCPP_INFO(rclcpp::get_logger("ArmbotInterface"), "Hardware activated.");
103:   return CallbackReturn::SUCCESS;
104: }
105:
106: CallbackReturn ArmbotInterface::on_deactivate(const rclcpp_lifecycle::State&)
107: {
108:   RCLCPP_INFO(rclcpp::get_logger("ArmbotInterface"), "Deactivating hardware...");
109:
110:   if (arduino_.IsOpen())
111:   {
112:     try
113:     {
114:       arduino_.Close();
115:     }
116:     catch (const std::exception& e)
117:     {
118:       RCLCPP_FATAL_STREAM(rclcpp::get_logger("ArmbotInterface"),
119:         "Exception while closing port " << port_ << ": " << e.what());
120:     }
121:   }
122:
123:   RCLCPP_INFO(rclcpp::get_logger("ArmbotInterface"), "Hardware deactivated.");
124:   return CallbackReturn::SUCCESS;
125: }
126:
127: hardware_interface::return_type ArmbotInterface::read(const rclcpp::Time&, const rclcpp::Duration&)
128: {
129:   // Simulate perfect execution by mirroring commands to states
130:   position_states_ = position_commands_;
131:   return hardware_interface::return_type::OK;
132: }
133:
134: hardware_interface::return_type ArmbotInterface::write(const rclcpp::Time&, const rclcpp::Duration&, const std::vector<float>& commands)
135: {
136:   if (position_commands_ == prev_position_commands_)
137:   {
138:     return hardware_interface::return_type::OK; // No change in commands, skip sending
139:   }
140:
141:   std::string msg;
142:   msg += format_command("b", static_cast<int>(((position_commands_[0] + M_PI / 2) * 180) / M_PI));
143:   msg += format_command("s", 180 - static_cast<int>(((position_commands_[1] + M_PI / 2) * 180) / M_PI));
144:   msg += format_command("e", static_cast<int>(((position_commands_[2] + M_PI / 2) * 180) / M_PI));
145:   msg += format_command("w", static_cast<int>(((position_commands_[3] + M_PI / 2) * 180) / M_PI));
146:   msg += format_command("r", static_cast<int>(((position_commands_[4] + M_PI / 2) * 180) / M_PI));
147:   msg += format_command("g", static_cast<int>((-position_commands_[5] * 180) / (M_PI / 2)));
148:
149:   try
150:   {
151:     RCLCPP_INFO_STREAM(rclcpp::get_logger("ArmbotInterface"), "Sending command: " << msg);
152:     arduino_.Write(msg);
153:   }
154:   catch (const std::exception& e)

```

```
155: {
156:     RCLCPP_ERROR_STREAM(rclcpp::get_logger("ArmbotInterface"),
157:         "Failed to send message to port " << port_ << ": " << e.what());
158:     return hardware_interface::return_type::ERROR;
159: }
160:
161: prev_position_commands_ = position_commands_;
162: return hardware_interface::return_type::OK;
163: }
164:
165: } // namespace armbot_controller
166:
167: PLUGINLIB_EXPORT_CLASS(armbot_controller::ArmbotInterface, hardware_interface::SystemInter
168:
```

File: slider_control.cpp (Other)

```
1: #include <rclcpp/rclcpp.hpp>
2: #include <sensor_msgs/msg/joint_state.hpp>
3: #include <trajectory_msgs/msg/joint_trajectory.hpp>
4: #include <trajectory_msgs/msg/joint_trajectory_point.hpp>
5: #include <vector>
6: #include <string>
7: #include <chrono>
8:
9: using namespace std::chrono_literals;
10: using std::placeholders::_1;
11:
12: class SliderControl : public rclcpp::Node
13: {
14: public:
15:     SliderControl() : Node("slider_control"),
16:                     arm_joint_names_{"joint_1", "joint_2", "joint_3", "joint_4", "joint_5"},
17:                     gripper_joint_names_{"joint_6"}
18:     {
19:         sub_ = create_subscription<sensor_msgs::msg::JointState>(
20:             "joint_commands", 10, std::bind(&SliderControl::sliderCallback, this, _1));
21:         arm_pub_ = create_publisher<trajectory_msgs::msg::JointTrajectory>("arm_controller/joint_commands");
22:         gripper_pub_ = create_publisher<trajectory_msgs::msg::JointTrajectory>("gripper_controller/joint_commands");
23:         RCLCPP_INFO(get_logger(), "Slider Control Node started");
24:     }
25:
26: private:
27:     rclcpp::Subscription<sensor_msgs::msg::JointState>::SharedPtr sub_;
28:     rclcpp::Publisher<trajectory_msgs::msg::JointTrajectory>::SharedPtr arm_pub_;
29:     rclcpp::Publisher<trajectory_msgs::msg::JointTrajectory>::SharedPtr gripper_pub_;
30:     std::vector<std::string> arm_joint_names_;
31:     std::vector<std::string> gripper_joint_names_;
32:
33:     void sliderCallback(const sensor_msgs::msg::JointState &msg) const
34:     {
35:         if (msg.position.size() < arm_joint_names_.size() + gripper_joint_names_.size())
36:         {
37:             RCLCPP_WARN(get_logger(), "Received joint state message with insufficient positions.");
38:             return;
39:         }
40:
41:         trajectory_msgs::msg::JointTrajectory arm_command, gripper_command;
42:         arm_command.joint_names = arm_joint_names_;
43:         gripper_command.joint_names = gripper_joint_names_;
44:
45:         trajectory_msgs::msg::JointTrajectoryPoint arm_goal, gripper_goal;
46:
47:         // Copy the positions for arm joints
48:         arm_goal.positions.insert(arm_goal.positions.end(), msg.position.begin(), msg.position.begin() + arm_joint_names_.size());
49:         // Copy the position for the gripper joint
50:         gripper_goal.positions.push_back(msg.position.at(arm_joint_names_.size())); // Assumes
51:
```

```
52:     arm_command.points.push_back(arm_goal);
53:     gripper_command.points.push_back(gripper_goal);
54:
55:     arm_pub_->publish(arm_command);
56:     gripper_pub_->publish(gripper_command);
57: }
58: };
59:
60: int main(int argc, char* argv[])
61: {
62:     rclcpp::init(argc, argv);
63:     auto node = std::make_shared<SliderControl>();
64:     rclcpp::spin(node);
65:     rclcpp::shutdown();
66:     return 0;
67: }
68:
```


src\armbot_description

File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.ros.org/xml-schemas/package_format3.xsd"?>
3: <package format="3">
4:   <name>armbot_description</name>
5:   <version>0.0.0</version>
6:   <description>Armbot Description package for ROS2 Jazzy and Gazebo Sim</description>
7:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
8:   <license>Apache 2.0</license>
9:
10:  <buildtool_depend>ament_cmake</buildtool_depend>
11:
12:  <exec_depend>robot_state_publisher</exec_depend>
13:  <exec_depend>urdf</exec_depend>
14:  <exec_depend>joint_state_publisher_gui</exec_depend>
15:  <exec_depend>rviz2</exec_depend>
16:  <exec_depend>xacro</exec_depend>
17:  <exec_depend>ros_gz_sim</exec_depend>
18:  <exec_depend>ros2launch</exec_depend>
19:  <exec_depend>ros_gz_bridge</exec_depend>
20:  <exec_depend>gz_ros2_control</exec_depend> <!-- Direct dependency for Jazzy -->
21:
22:  <test_depend>ament_lint_auto</test_depend>
23:  <test_depend>ament_lint_common</test_depend>
24:
25:  <export>
26:    <build_type>ament_cmake</build_type>
27:  </export>
28: </package>
29:
```

src\armbot_description\launch

[File: display.launch.py \(Launch File\)](#)

```
1: import os
2: from ament_index_python.packages import get_package_share_directory
3: from launch import LaunchDescription
4: from launch.actions import DeclareLaunchArgument
5: from launch.substitutions import Command, LaunchConfiguration
6: from launch_ros.actions import Node
7: from launch_ros.parameter_descriptions import ParameterValue
8:
9: def generate_launch_description():
10:     # Get package directory for armbot_description
11:     armbot_description_dir = get_package_share_directory('armbot_description')
12:
13:     # Declare arguments for model file path and use_sim_time
14:     model_arg = DeclareLaunchArgument(
15:         name='model',
16:         default_value=os.path.join(armbot_description_dir, 'urdf', 'armbot.urdf.xacro'),
17:         description='Absolute path to robot URDF/Xacro file'
18:     )
19:     use_sim_time_arg = DeclareLaunchArgument(
20:         name='use_sim_time',
21:         default_value='true',
22:         description='Use simulation time if true'
23:     )
24:
25:     # Robot description as a parameter
26:     robot_description = ParameterValue(
27:         Command(['xacro ', LaunchConfiguration('model')]), value_type=str
28:     )
29:
30:     # Node for publishing robot state
31:     robot_state_publisher_node = Node(
32:         package='robot_state_publisher',
33:         executable='robot_state_publisher',
34:         parameters=[{'robot_description': robot_description, 'use_sim_time': LaunchConfiguration('use_sim_time')}],
35:     )
36:
37:     # Node for joint state publisher GUI
38:     joint_state_publisher_gui_node = Node(
39:         package='joint_state_publisher_gui',
40:         executable='joint_state_publisher_gui'
41:     )
42:
43:     # Node for launching RViz with a pre-configured display file
44:     rviz_node = Node(
45:         package='rviz2',
46:         executable='rviz2',
47:         name='rviz2',
48:         output='screen',
```

```
49:         arguments=['-d', os.path.join(armbot_description_dir, 'rviz', 'display.rviz')],
50:     )
51:
52:     return LaunchDescription([
53:         model_arg,
54:         use_sim_time_arg,
55:         joint_state_publisher_gui_node,
56:         robot_state_publisher_node,
57:         rviz_node
58:     ])
59:
```

[File: gazebo.launch.py \(Launch File\)](#)

```
1: import os
2: from pathlib import Path
3: from ament_index_python.packages import get_package_share_directory
4: from launch import LaunchDescription
5: from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, SetEnvironmentVariable
6: from launch.substitutions import Command, LaunchConfiguration
7: from launch.launch_description_sources import PythonLaunchDescriptionSource
8: from launch_ros.actions import Node
9: from launch_ros.parameter_descriptions import ParameterValue
10:
11: def generate_launch_description():
12:     # Get package directory for armbot_description
13:     armbot_description_dir = get_package_share_directory('armbot_description')
14:
15:     # Declare arguments for model file path and use_sim_time
16:     model_arg = DeclareLaunchArgument(
17:         name='model',
18:         default_value=os.path.join(armbot_description_dir, 'urdf', 'armbot.urdf.xacro'),
19:         description='Absolute path to robot URDF/Xacro file'
20:     )
21:     use_sim_time_arg = DeclareLaunchArgument(
22:         name='use_sim_time',
23:         default_value='true',
24:         description='Use simulation time if true'
25:     )
26:
27:     # Set environment variable for Gazebo resource path
28:     gazebo_resource_path = SetEnvironmentVariable(
29:         name='GZ_SIM_RESOURCE_PATH',
30:         value=[str(Path(armbot_description_dir).parent.resolve())]
31:     )
32:
33:     # Generate robot description as a parameter
34:     robot_description = ParameterValue(
35:         Command(['xacro ', LaunchConfiguration('model')]), value_type=str
36:     )
37:
38:     # Node for robot state publisher
39:     robot_state_publisher_node = Node(
40:         package='robot_state_publisher',
41:         executable='robot_state_publisher',
42:         parameters=[{'robot_description': robot_description, 'use_sim_time': LaunchConfiguration('use_sim_time')}]
43:     )
44:
45:     # Include the Gazebo launch file
46:     gazebo = IncludeLaunchDescription(
47:         PythonLaunchDescriptionSource([
48:             os.path.join(get_package_share_directory('ros_gz_sim'), 'launch', 'gz_sim.launch.py'),
49:         ]),
50:         launch_arguments=[('gz_args', ['-v 4 -r empty.sdf --physics-engine gz-physics-bullet'])]
51:     )
```

```

52:
53:     # Node for spawning the robot entity in Gazebo
54:     gz_spawn_entity = Node(
55:         package='ros_gz_sim',
56:         executable='create',
57:         output='screen',
58:         arguments=['-topic', 'robot_description', '-name', 'armbot']
59:     )
60:
61:     # Node for bridging topics between ROS 2 and Gazebo
62:     gz_ros2_bridge = Node(
63:         package='ros_gz_bridge',
64:         executable='parameter_bridge',
65:         arguments=[
66:             '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
67:             '/image_raw@sensor_msgs/msg/Image[gz.msgs.Image',
68:             '/camera_info@sensor_msgs/msg/CameraInfo[gz.msgs.CameraInfo'
69:         ],
70:         output='screen'
71:     )
72:
73:     return LaunchDescription([
74:         model_arg,
75:         use_sim_time_arg,
76:         gazebo_resource_path,
77:         robot_state_publisher_node,
78:         gazebo,
79:         gz_spawn_entity,
80:         gz_ros2_bridge
81:     ])
82:

```

src\armbot_description\urdf

File: armbot.urdf.xacro (URDF File)

```
1: <?xml version="1.0"?>
2: <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="armbot">
3:
4:     <!-- Load Simulated or Real Hardware Interfaces -->
5:     <xacro:arg name="is_sim" default="true"/>
6:
7:     <!-- Include Gazebo Parameters -->
8:     <xacro:include filename="$(find armbot_description)/urdf/armbot_gazebo.xacro" />
9:
10:    <!-- Include ros2_control Plugins -->
11:    <xacro:include filename="$(find armbot_description)/urdf/armbot_ros2_control.xacro" />
12:
13:    <!-- Useful XACRO Variables (Properties) -->
14:    <xacro:property name="PI" value="3.14159265359" />
15:    <xacro:property name="effort" value="1000.0" />
16:    <xacro:property name="velocity" value="1.0" />
17:
18:    <!-- Useful Macros -->
19:    <xacro:macro name="default_inertial" params="mass">
20:        <inertial>
21:            <origin xyz="0 0 0" rpy="0 0 0" />
22:            <mass value="{mass}" />
23:            <inertia ixx="1.0" ixy="0.0" ixz="0.0"
24:                iyy="1.0" iyz="0.0"
25:                izz="1.0" />
26:        </inertial>
27:    </xacro:macro>
28:
29:    <xacro:macro name="default_transmission" params="number">
30:        <transmission name="transmission_${number}">
31:            <plugin>transmission_interface/SimpleTransmission</plugin>
32:            <actuator name="motor_${number}" role="actuator1" />
33:            <joint name="joint_${number}" role="joint1">
34:                <mechanical_reduction>1.0</mechanical_reduction>
35:            </joint>
36:        </transmission>
37:    </xacro:macro>
38:
39:    <!-- Links -->
40:    <link name="world" />
41:
42:    <link name="base_link">
43:        <xacro:default_inertial mass="1.0" />
44:        <visual>
45:            <origin rpy="0 0 0" xyz="0 0 0.01" />
46:            <geometry>
47:                <mesh filename="package://armbot_description/meshes/base_cylinder.stl"
48:                    scale="0.0001 0.0001 0.0001" />
```

```

49:         </geometry>
50:         <material name="black">
51:             <color rgba="0 0 0 1" />
52:         </material>
53:     </visual>
54:     <collision>
55:         <origin rpy="0 0 0" xyz="0 0 0.01" />
56:         <geometry>
57:             <mesh filename="package://armbot_description/meshes/base_cylinder.stl"
58:                 scale="0.0001 0.0001 0.0001" />
59:         </geometry>
60:     </collision>
61: </link>
62:
63: <link name="platform">
64:     <xacro:default_inertial mass="0.5" />
65:     <visual>
66:         <origin rpy="0 0 0" xyz="0 0 0" />
67:         <geometry>
68:             <mesh filename="package://armbot_description/meshes/top_plate.stl"
69:                 scale="0.001 0.001 0.001" />
70:         </geometry>
71:         <material name="black" />
72:     </visual>
73:     <collision>
74:         <origin rpy="0 0 0" xyz="0 0 0" />
75:         <geometry>
76:             <mesh filename="package://armbot_description/meshes/top_plate.stl"
77:                 scale="0.001 0.001 0.001" />
78:         </geometry>
79:     </collision>
80: </link>
81:
82: <link name="lower_arm">
83:     <xacro:default_inertial mass="0.5" />
84:     <visual>
85:         <origin rpy="0 0 0" xyz="0 0 0" />
86:         <geometry>
87:             <mesh filename="package://armbot_description/meshes/lower_arm.stl"
88:                 scale="0.001 0.001 0.001" />
89:         </geometry>
90:         <material name="black" />
91:     </visual>
92:     <collision>
93:         <origin rpy="0 0 0" xyz="0 0 0" />
94:         <geometry>
95:             <mesh filename="package://armbot_description/meshes/lower_arm.stl"
96:                 scale="0.001 0.001 0.001" />
97:         </geometry>
98:     </collision>
99: </link>
100:
101:     <link name="top_arm">

```

```

102:     <xacro:default_inertial mass="0.5" />
103:     <visual>
104:         <origin rpy="0 1.59 0" xyz="0 0 0" />
105:         <geometry>
106:             <mesh filename="package://armbot_description/meshes/top_arm.stl"
107:                 scale="0.001 0.001 0.001" />
108:         </geometry>
109:         <material name="black" />
110:     </visual>
111:     <collision>
112:         <origin rpy="0 1.59 0" xyz="0 0 0" />
113:         <geometry>
114:             <mesh filename="package://armbot_description/meshes/top_arm.stl"
115:                 scale="0.001 0.001 0.001" />
116:         </geometry>
117:     </collision>
118: </link>
119:
120: <link name="wrist">
121:     <xacro:default_inertial mass="0.5" />
122:     <visual>
123:         <origin rpy="0 0 0" xyz="0 0 0" />
124:         <geometry>
125:             <mesh filename="package://armbot_description/meshes/wrist.stl"
126:                 scale="0.001 0.001 0.001" />
127:         </geometry>
128:         <material name="black" />
129:     </visual>
130:     <collision>
131:         <origin rpy="0 0 0" xyz="0 0 0" />
132:         <geometry>
133:             <mesh filename="package://armbot_description/meshes/wrist.stl"
134:                 scale="0.001 0.001 0.001" />
135:         </geometry>
136:     </collision>
137: </link>
138:
139: <link name="clawbase">
140:     <xacro:default_inertial mass="0.2" />
141:     <visual>
142:         <origin rpy="0 3.18 0" xyz="0 0 0" />
143:         <geometry>
144:             <mesh filename="package://armbot_description/meshes/clawbase.stl"
145:                 scale="0.001 0.001 0.001" />
146:         </geometry>
147:         <material name="white">
148:             <color rgba="1 1 1 1" />
149:         </material>
150:     </visual>
151:     <collision>
152:         <origin rpy="0 3.18 0" xyz="0 0 0" />
153:         <geometry>
154:             <mesh filename="package://armbot_description/meshes/clawbase.stl"

```



```

155:             scale="0.001 0.001 0.001" />
156:         </geometry>
157:     </collision>
158: </link>
159:
160: <link name="clawr">
161:     <xacro:default_inertial mass="0.1" />
162:     <visual>
163:         <origin rpy="0 0 0" xyz="0 0 0" />
164:         <geometry>
165:             <mesh filename="package://armbot_description/meshes/clawr.stl"
166:                 scale="0.001 0.001 0.001" />
167:         </geometry>
168:         <material name="white" />
169:     </visual>
170:     <collision>
171:         <origin rpy="0 0 0" xyz="0 0 0" />
172:         <geometry>
173:             <mesh filename="package://armbot_description/meshes/clawr.stl"
174:                 scale="0.001 0.001 0.001" />
175:         </geometry>
176:     </collision>
177: </link>
178:
179: <link name="clawl">
180:     <xacro:default_inertial mass="0.1" />
181:     <visual>
182:         <origin rpy="0 0 0" xyz="0 0 0" />
183:         <geometry>
184:             <mesh filename="package://armbot_description/meshes/clawl.stl"
185:                 scale="0.001 0.001 0.001" />
186:         </geometry>
187:         <material name="white" />
188:     </visual>
189:     <collision>
190:         <origin rpy="0 0 0" xyz="0 0 0" />
191:         <geometry>
192:             <mesh filename="package://armbot_description/meshes/clawl.stl"
193:                 scale="0.001 0.001 0.001" />
194:         </geometry>
195:     </collision>
196: </link>
197:
198: <!-- Joints -->
199: <joint name="virtual_joint" type="fixed">
200:     <parent link="world" />
201:     <child link="base_link" />
202:     <origin xyz="0 0 0" rpy="0 0 0" />
203: </joint>
204:
205: <joint name="joint_1" type="revolute">
206:     <axis xyz="0 0 1" />
207:     <limit effort="{effort}" lower="1.56" upper="4.68" velocity="{velocity}" />

```

```

208:         <parent link="base_link" />
209:         <child link="platform" />
210:         <origin rpy="0 0 1.5" xyz="0 0 0.48" />
211:     </joint>
212:
213:     <joint name="joint_2" type="revolute">
214:         <axis xyz="1 0 0" />
215:         <limit effort="${effort}" lower="0.0" upper="3.12" velocity="${velocity}" />
216:         <parent link="platform" />
217:         <child link="lower_arm" />
218:         <origin xyz="0 0.19 0.28" />
219:     </joint>
220:
221:     <joint name="joint_3" type="revolute">
222:         <axis xyz="1 0 0" />
223:         <limit effort="${effort}" lower="-1.7" upper="1.7" velocity="${velocity}" />
224:         <parent link="lower_arm" />
225:         <child link="top_arm" />
226:         <origin xyz="-0.25 1.0 0" />
227:     </joint>
228:
229:     <joint name="joint_4" type="revolute">
230:         <axis xyz="1 0 0" />
231:         <limit effort="${effort}" lower="-1.7" upper="1.7" velocity="${velocity}" />
232:         <parent link="top_arm" />
233:         <child link="wrist" />
234:         <origin xyz="0.11 1.25 0" />
235:     </joint>
236:
237:     <joint name="joint_5" type="revolute">
238:         <axis xyz="0 1 0" />
239:         <limit effort="${effort}" lower="-3.17" upper="0.0" velocity="${velocity}" />
240:         <parent link="wrist" />
241:         <child link="clawbase" />
242:         <origin xyz="0 0.63 0" />
243:     </joint>
244:
245:     <joint name="joint_6" type="revolute">
246:         <axis xyz="0 0 1" />
247:         <limit effort="${effort}" lower="-0.5" upper="0.0" velocity="${velocity}" />
248:         <parent link="clawbase" />
249:         <child link="clawr" />
250:         <origin rpy="0 0 -0.31" xyz="0.11 0.34 0" />
251:     </joint>
252:
253:     <joint name="joint_7" type="revolute">
254:         <axis xyz="0 0 1" />
255:         <limit effort="${effort}" lower="0" upper="-0.5" velocity="${velocity}" />
256:         <parent link="clawbase" />
257:         <child link="clawl" />
258:         <origin rpy="0 0 0.22" xyz="-0.16 0.30 0" />
259:         <mimic joint="joint_6" multiplier="-1" offset="0" />
260:     </joint>

```

```
261:
262:     <!-- Transmissions -->
263:     <xacro:default_transmission number="1" />
264:     <xacro:default_transmission number="2" />
265:     <xacro:default_transmission number="3" />
266:     <xacro:default_transmission number="4" />
267:     <xacro:default_transmission number="5" />
268:     <xacro:default_transmission number="6" />
269:     <xacro:default_transmission number="7" />
270:
271: </robot>
```

File: armbot_gazebo.xacro (URDF File)

```
1: <?xml version="1.0"?>
2: <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="armbot">
3:
4:   <!-- Gazebo plugin setup for ROS 2 Jazzy -->
5:   <gazebo>
6:     <plugin filename="gz_ros2_control-system" name="gz_ros2_control::GazeboSimROS2Contro
7:       <parameters>$(find armbot_controller)/config/armbot_controllers.yaml</parameters>
8:     </plugin>
9:     <plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
10:       <render_engine>ogre2</render_engine>
11:     </plugin>
12:   </gazebo>
13:
14:   <!-- Camera sensor configuration -->
15:   <gazebo reference="camera">
16:     <sensor name="camera" type="camera">
17:       <always_on>true</always_on>
18:       <update_rate>30.0</update_rate>
19:       <visualize>true</visualize>
20:       <topic>/image_raw</topic>
21:       <gz_frame_id>/camera</gz_frame_id>
22:       <camera name="camera">
23:         <horizontal_fov>1.21126</horizontal_fov>
24:         <image>
25:           <width>2304</width>
26:           <height>1296</height>
27:           <format>R8G8B8</format>
28:         </image>
29:         <distortion>
30:           <k1>0.0</k1>
31:           <k2>0.0</k2>
32:           <k3>0.0</k3>
33:           <p1>0.0</p1>
34:           <p2>0.0</p2>
35:           <center>0.5 0.5</center>
36:         </distortion>
37:       </camera>
38:     </sensor>
39:   </gazebo>
40:
41: </robot>
42:
```

File: armbot_ros2_control.xacro (URDF File)

```
1: <?xml version="1.0"?>
2: <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="armbot">
3:
4:   <!-- ROS 2 Control System -->
5:   <ros2_control name="RobotSystem" type="system">
6:     <!-- Define common properties -->
7:     <xacro:property name="PI" value="3.14159265359" />
8:
9:     <!-- Hardware interface setup -->
10:    <xacro:macro name="hardware_interface" params="is_sim plugin_name port">
11:      <xacro:if value="${is_sim}">
12:        <hardware>
13:          <plugin>${plugin_name}</plugin>
14:        </hardware>
15:      </xacro:if>
16:      <xacro:unless value="${is_sim}">
17:        <hardware>
18:          <plugin>armbot_controller/ArmbotInterface</plugin>
19:          <param name="port">${port}</param>
20:        </hardware>
21:      </xacro:unless>
22:    </xacro:macro>
23:
24:    <!-- Invoke hardware interface for simulation or real robot -->
25:    <xacro:hardware_interface is_sim="$(arg is_sim)" plugin_name="gz_ros2_control/Gazebo" />
26:
27:    <!-- Common joint interface macro -->
28:    <xacro:macro name="joint_interface" params="joint_name min_angle max_angle">
29:      <joint name="${joint_name}">
30:        <command_interface name="position">
31:          <param name="min">${min_angle}</param>
32:          <param name="max">${max_angle}</param>
33:        </command_interface>
34:        <state_interface name="position" />
35:      </joint>
36:    </xacro:macro>
37:
38:    <!-- Define joint interfaces -->
39:    <xacro:joint_interface joint_name="joint_1" min_angle="-${PI / 2}" max_angle="${PI / 2}" />
40:    <xacro:joint_interface joint_name="joint_2" min_angle="-${PI / 2}" max_angle="${PI / 2}" />
41:    <xacro:joint_interface joint_name="joint_3" min_angle="-${PI / 2}" max_angle="${PI / 2}" />
42:    <xacro:joint_interface joint_name="joint_4" min_angle="-${PI / 2}" max_angle="${PI / 2}" />
43:    <xacro:joint_interface joint_name="joint_5" min_angle="-${PI / 2}" max_angle="${PI / 2}" />
44:    <xacro:joint_interface joint_name="joint_6" min_angle="-${PI / 2}" max_angle="0.0" />
45:
46:    <!-- Mimic joint configuration -->
47:    <joint name="joint_7">
48:      <param name="mimic">joint_6</param>
49:      <param name="multiplier">-1</param>
50:    </joint>
51:
```

```
52:     </ros2_control>
53:
54: </robot>
55:
```

src\armbot_firmware

File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://w
3: <package format="3">
4:   <name>armbot_firmware</name>
5:   <version>0.0.0</version>
6:   <description>Arduino Firmware</description>
7:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
8:   <license>Apache 2.0</license>
9:
10:   <buildtool_depend>ament_cmake</buildtool_depend>
11:   <buildtool_depend>ament_cmake_python</buildtool_depend>
12:
13:   <depend>rclcpp</depend>
14:   <depend>rclpy</depend>
15:   <depend>libserial-dev</depend>
16:   <depend>std_msgs</depend>
17:
18:   <exec_depend>python3-serial</exec_depend>
19:
20:   <test_depend>ament_lint_auto</test_depend>
21:   <test_depend>ament_lint_common</test_depend>
22:
23:   <export>
24:     <build_type>ament_cmake</build_type>
25:   </export>
26: </package>
27:
```

src\armbot_firmware\armbot_firmware

File: [__init__.py](#) (Other)

1:

[File: simple_serial_receiver.py \(Other\)](#)

```
1: #!/usr/bin/env python3
2: import serial
3: import rclpy
4: from rclpy.node import Node
5: from std_msgs.msg import String
6:
7:
8: class SimpleSerialReceiver(Node):
9:     def __init__(self):
10:         super().__init__("simple_serial_receiver")
11:
12:         self.declare_parameter("port", "/dev/ttyUSB0")
13:         self.declare_parameter("baudrate", 115200)
14:
15:         self.port_ = self.get_parameter("port").value
16:         self.baudrate_ = self.get_parameter("baudrate").value
17:
18:         self.pub_ = self.create_publisher(String, "serial_receiver", 10)
19:         self.arduino_ = serial.Serial(port=self.port_, baudrate=self.baudrate_, timeout=0.1)
20:
21:         self.frequency_ = 0.01
22:         self.timer_ = self.create_timer(self.frequency_, self.timerCallback)
23:
24:     def timerCallback(self):
25:         if rclpy.ok() and self.arduino_.is_open:
26:             data = self.arduino_.readline()
27:
28:             try:
29:                 data.decode("utf-8")
30:             except:
31:                 return
32:
33:             msg = String()
34:             msg.data = str(data)
35:             self.pub_.publish(msg)
36:
37:
38: def main():
39:     rclpy.init()
40:
41:     simple_serial_receiver = SimpleSerialReceiver()
42:     rclpy.spin(simple_serial_receiver)
43:
44:     simple_serial_receiver.destroy_node()
45:     rclpy.shutdown()
46:
47:
48: if __name__ == '__main__':
49:     main()
```

[File: simple_serial_transmitter.py \(Other\)](#)

```
1: #!/usr/bin/env python3
2: import rclpy
3: from rclpy.node import Node
4: from std_msgs.msg import String
5: import serial
6:
7:
8: class SimpleSerialTransmitter(Node):
9:     def __init__(self):
10:         super().__init__("simple_serial_transmitter")
11:
12:         self.declare_parameter("port", "/dev/ttyUSB0")
13:         self.declare_parameter("baudrate", 115200)
14:
15:         self.port_ = self.get_parameter("port").value
16:         self.baudrate_ = self.get_parameter("baudrate").value
17:
18:         self.sub_ = self.create_subscription(String, "serial_transmitter", self.msgCallback, 1)
19:         self.sub_
20:         self.arduino_ = serial.Serial(port=self.port_, baudrate=self.baudrate_, timeout=0.1)
21:
22:     def msgCallback(self, msg):
23:         self.get_logger().info("New message received, publishing on serial: %s" % self.arduino_.write(msg.data.encode("utf-8")))
24:         self.arduino_.write(msg.data.encode("utf-8"))
25:
26:
27: def main():
28:     rclpy.init()
29:
30:     simple_serial_transmitter = SimpleSerialTransmitter()
31:     rclpy.spin(simple_serial_transmitter)
32:
33:     simple_serial_transmitter.destroy_node()
34:     rclpy.shutdown()
35:
36:
37: if __name__ == '__main__':
38:     main()
```

src\armbot_firmware\firmware\robot_control

File: robot_control.ino (Other)

```
1: #include <Servo.h>
2:
3: // Declare the Arduino pin where each servo is connected
4: #define SERVO_BASE_PIN 8
5: #define SERVO_SHOULDER_PIN 9
6: #define SERVO_ELBOW_PIN 10
7: #define SERVO_WRISTE_PITCH_PIN 11
8: #define SERVO_WRISTE_ROLL_PIN 12
9: #define SERVO_GRIPPER_PIN 7
10:
11: // Define the start configuration of the joints
12: #define BASE_START 90
13: #define SHOULDER_START 90
14: #define ELBOW_START 90
15: #define WRISTE_PITCH_START 90
16: #define WRISTE_ROLL_START 90
17: #define GRIPPER_START 0
18:
19: // Register the servo motors of each joint
20: Servo base;
21: Servo shoulder;
22: Servo elbow;
23: Servo wriste_pitch;
24: Servo wriste_roll;
25: Servo gripper;
26:
27: uint8_t idx = 0;
28: uint8_t val_idx = 0;
29: char value[4] = "000";
30:
31:
32: /*
33:  * This function moves a given servo smoothly from a given start position to a given end po
34:  * The movement can be both clockwise or counterclockwise based on the values assigned to
35:  * the start position and end position
36:  */
37: void reach_goal(Servo& motor, int goal){
38:     if(goal>=motor.read()){
39:         // goes from the start point degrees to the end point degrees
40:         for (int pos = motor.read(); pos <= goal; pos += 1) {
41:             motor.write(pos);
42:             delay(5);
43:         }
44:     } else{
45:         // goes from the end point degrees to the start point degrees
46:         for (int pos = motor.read(); pos >= goal; pos -= 1) {
47:             motor.write(pos);
48:             delay(5);
```

```

49:     }
50: }
51: }
52:
53: void setup() {
54:     // Attach and Initialize each Servo to the Arduino pin where it is connected
55:     base.attach(SERVO_BASE_PIN);
56:     shoulder.attach(SERVO_SHOULDER_PIN);
57:     elbow.attach(SERVO_ELBOW_PIN);
58:     wriste_pitch.attach(SERVO_WRISTE_PITCH_PIN);
59:     wriste_roll.attach(SERVO_WRISTE_ROLL_PIN);
60:     gripper.attach(SERVO_GRIPPER_PIN);
61:
62:     // Set a common start point for each joint
63:     // This way, the start status of each joint is known
64:     base.write(BASE_START);
65:     shoulder.write(SHOULDER_START);
66:     elbow.write(ELBOW_START);
67:     wriste_pitch.write(WRISTE_PITCH_START);
68:     wriste_roll.write(WRISTE_ROLL_START);
69:     gripper.write(GRIPPER_START);
70:
71:     // Start the Serial communication with ROS
72:     Serial.begin(115200);
73:     Serial.setTimeout(1);
74: }
75:
76: void loop() {
77:     if (Serial.available())
78:     {
79:         char chr = Serial.read();
80:
81:         // base motor
82:         if(chr == 'b')
83:         {
84:             idx = 0;
85:             val_idx = 0;
86:         }
87:         // shoulder motor
88:         else if(chr == 's')
89:         {
90:             idx = 1;
91:             val_idx = 0;
92:         }
93:         // elbow motor
94:         else if(chr == 'e')
95:         {
96:             idx = 2;
97:             val_idx = 0;
98:         }
99:         //wrist_pitch motor
100:         else if(chr == 'w')
101:         {

```

```

102:     idx = 3;
103:     val_idx = 0;
104: }
105: //wrist_roll motor
106: else if(chr == 'r')
107: {
108:     idx = 4;
109:     val_idx = 0;
110: }
111: // gripper motor
112: else if(chr == 'g')
113: {
114:     idx = 5;
115:     val_idx = 0;
116: }
117: // Separator
118: else if(chr == ',')
119: {
120:     int val = atoi(value);
121:     if(idx == 0)
122:     {
123:         reach_goal(base, val);
124:     }
125:     else if(idx == 1)
126:     {
127:         reach_goal(shoulder, val);
128:     }
129:     else if(idx == 2)
130:     {
131:         reach_goal(elbow, val);
132:     }
133:     else if(idx == 3)
134:     {
135:         reach_goal(wrist_pitch, val);
136:     }
137:     else if(idx == 4)
138:     {
139:         reach_goal(wrist_roll, val);
140:     }
141:     else if(idx == 5)
142:     {
143:         reach_goal(gripper, val);
144:     }
145:
146:     // reset the angle
147:     value[0] = '0';
148:     value[1] = '0';
149:     value[2] = '0';
150:     value[3] = '\0';
151: }
152: // Plain number
153: else
154: {

```

```
155:         value[val_idx] = chr;
156:         val_idx++;
157:     }
158: }
159: }
160:
```

src\armbot_firmware\firmware\simple_serial_receiver

[File: simple_serial_receiver.ino \(Other\)](#)

```
1: #define LED_PIN 13
2:
3: void setup() {
4:   pinMode(LED_PIN, OUTPUT);
5:   digitalWrite(LED_PIN, LOW);
6:
7:   Serial.begin(115200);
8:   Serial.setTimeout(1);
9: }
10:
11: void loop() {
12:   if (Serial.available())
13:   {
14:     int x = Serial.readString().toInt();
15:     if(x == 0)
16:     {
17:       // turn off the led
18:       digitalWrite(LED_PIN, LOW);
19:     }
20:     else
21:     {
22:       // turn on the led
23:       digitalWrite(LED_PIN, HIGH);
24:     }
25:   }
26:   delay(0.1);
27: }
28:
```

src\armbot_firmware\firmware\simple_serial_transmitter

[File: simple_serial_transmitter.ino \(Other\)](#)

```
1: int x = 0;
2:
3: void setup() {
4:   Serial.begin(115200);
5:   Serial.setTimeout(1);
6: }
7:
8: void loop() {
9:   Serial.println(x);
10:  x++;
11:  delay(0.1);
12: }
13:
```


src\armbot_firmware\firmware\simple_servo_control

File: simple_servo_control.ino (Other)

```
1: #include <Servo.h>
2:
3: // Register a servo motor
4: Servo motor;
5:
6: void setup() {
7:   // Attach and Initialize each Servo to the Arduino pin where it is connected
8:   motor.attach(8);
9:
10:  // Set a known starting point for the motor
11:  motor.write(90);
12:
13:  // Start the Serial communication with ROS
14:  Serial.begin(115200);
15:  Serial.setTimeout(1);
16: }
17:
18: void loop() {
19:   if (Serial.available())
20:   {
21:     int angle = Serial.readString().toInt();
22:     motor.write(angle);
23:   }
24:   delay(0.1);
25: }
26:
```

src\armbot_firmware\src

File: simple_serial_receiver.cpp (Other)

```
1: #include <rclcpp/rclcpp.hpp>
2: #include <std_msgs/msg/string.hpp>
3:
4: #include <chrono>
5:
6: #include <libserial/SerialPort.h>
7:
8: using namespace std::chrono_literals;
9:
10:
11: class SimpleSerialReceiver : public rclcpp::Node
12: {
13: public:
14:     SimpleSerialReceiver() : Node("simple_serial_receiver")
15:     {
16:         declare_parameter<std::string>("port", "/dev/ttyUSB0");
17:
18:         port_ = get_parameter("port").as_string();
19:
20:         arduino_.Open(port_);
21:         arduino_.SetBaudRate(LibSerial::BaudRate::BAUD_115200);
22:
23:         pub_ = create_publisher<std_msgs::msg::String>("serial_receiver", 10);
24:         timer_ = create_wall_timer(0.01s, std::bind(&SimpleSerialReceiver::timerCallback, this))
25:     }
26:
27:     ~SimpleSerialReceiver()
28:     {
29:         arduino_.Close();
30:     }
31:
32:     void timerCallback()
33:     {
34:         if(rclcpp::ok() && arduino_.IsDataAvailable())
35:         {
36:             auto message = std_msgs::msg::String();
37:             arduino_.ReadLine(message.data);
38:             pub_->publish(message);
39:         }
40:     }
41:
42: private:
43:     rclcpp::Publisher<std_msgs::msg::String>::SharedPtr pub_;
44:     rclcpp::TimerBase::SharedPtr timer_;
45:     std::string port_;
46:     LibSerial::SerialPort arduino_;
47: };
48:
```

```
49:
50: int main(int argc, char* argv[])
51: {
52:     rclcpp::init(argc, argv);
53:     auto node = std::make_shared<SimpleSerialReceiver>();
54:     rclcpp::spin(node);
55:     rclcpp::shutdown();
56:     return 0;
57: }
58:
```

[File: simple_serial_transmitter.cpp \(Other\)](#)

```
1: #include <rclcpp/rclcpp.hpp>
2: #include <std_msgs/msg/string.hpp>
3:
4: #include <libserial/SerialPort.h>
5:
6: using std::placeholders::_1;
7:
8: class SimpleSerialTransmitter : public rclcpp::Node
9: {
10: public:
11:   SimpleSerialTransmitter() : Node("simple_serial_transmitter")
12:   {
13:     declare_parameter<std::string>("port", "/dev/ttyUSB0");
14:
15:     port_ = get_parameter("port").as_string();
16:
17:     sub_ = create_subscription<std_msgs::msg::String>(
18:       "serial_transmitter", 10, std::bind(&SimpleSerialTransmitter::msgCallback, this, _1
19:
20:     arduino_.Open(port_);
21:     arduino_.SetBaudRate(LibSerial::BaudRate::BAUD_115200);
22:   }
23:
24:   ~SimpleSerialTransmitter()
25:   {
26:     arduino_.Close();
27:   }
28:
29: private:
30:   rclcpp::Subscription<std_msgs::msg::String>::SharedPtr sub_;
31:   std::string port_;
32:   LibSerial::SerialPort arduino_;
33:
34:   void msgCallback(const std_msgs::msg::String &msg)
35:   {
36:     RCLCPP_INFO_STREAM(this->get_logger(), "New message received, publishing on serial: " <
37:     arduino_.Write(msg.data);
38:   }
39: };
40:
41:
42: int main(int argc, char * argv[])
43: {
44:   rclcpp::init(argc, argv);
45:   rclcpp::spin(std::make_shared<SimpleSerialTransmitter>());
46:   rclcpp::shutdown();
47:   return 0;
48: }
49:
```

src/armbot_moveit

File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://w
3: <package format="3">
4:   <name>armbot_moveit</name>
5:   <version>0.0.0</version>
6:   <description>Armbot MoveIt! 2 package</description>
7:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
8:   <license>Apache 2.0</license>
9:
10:   <buildtool_depend>ament_cmake</buildtool_depend>
11:
12:   <exec_depend>ros2launch</exec_depend>
13:   <exec_depend>rviz2</exec_depend>
14:   <exec_depend>moveit_configs_utils</exec_depend>
15:   <exec_depend>moveit_ros_move_group</exec_depend>
16:   <exec_depend>armbot_description</exec_depend>
17:
18:   <test_depend>ament_lint_auto</test_depend>
19:   <test_depend>ament_lint_common</test_depend>
20:
21:   <export>
22:     <build_type>ament_cmake</build_type>
23:   </export>
24: </package>
25:
```

src\armbot_moveit\config

[File: initial_positions.yaml \(Configuration File\)](#)

```
1: # Default initial positions for armbot's ros2_control fake system
2:
3: initial_positions:
4:   joint_1: 0
5:   joint_2: 0
6:   joint_3: 0
7:   joint_4: 0
8:   joint_5: 0
9:   joint_6: 0
```

File: joint_limits.yaml (Configuration File)

```
1: # joint_limits.yaml allows the dynamics properties specified in the URDF to be overwritten o
2:
3: # For beginners, we downscale velocity and acceleration limits.
4: # You can always specify higher scaling factors ( $\leq 1.0$ ) in your motion requests. # Increase
5: default_velocity_scaling_factor: 0.1
6: default_acceleration_scaling_factor: 0.1
7:
8: # Specific joint properties can be changed with the keys [max_position, min_position, max_ve
9: # Joint limits can be turned off with [has_velocity_limits, has_acceleration_limits]
10: joint_limits:
11:   joint_1:
12:     has_velocity_limits: true
13:     max_velocity: 10.0
14:     has_acceleration_limits: true
15:     max_acceleration: 5.0
16:   joint_2:
17:     has_velocity_limits: true
18:     max_velocity: 10.0
19:     has_acceleration_limits: true
20:     max_acceleration: 5.0
21:   joint_3:
22:     has_velocity_limits: true
23:     max_velocity: 10.0
24:     has_acceleration_limits: true
25:     max_acceleration: 5.0
26:   joint_4:
27:     has_velocity_limits: true
28:     max_velocity: 10.0
29:     has_acceleration_limits: true
30:     max_acceleration: 5.0
31:   joint_5:
32:     has_velocity_limits: true
33:     max_velocity: 10.0
34:     has_acceleration_limits: true
35:     max_acceleration: 5.0
36:   joint_6:
37:     has_velocity_limits: true
38:     max_velocity: 10.0
39:     has_acceleration_limits: true
40:     max_acceleration: 5.0
41:   joint_7:
42:     has_velocity_limits: true
43:     max_velocity: 10.0
44:     has_acceleration_limits: true
45:     max_acceleration: 5.0
46:
```

[File: kinematics.yaml \(Configuration File\)](#)

```
1: arm:
2:   kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
3:   kinematics_solver_search_resolution: 0.0050000000000000001
4:   kinematics_solver_timeout: 0.050000000000000001
5:   position_only_ik: True
```


[File: moveit_controllers.yaml \(Configuration File\)](#)

```
1: # MoveIt uses this configuration for controller management
2: moveit_controller_manager: moveit_simple_controller_manager/MoveItSimpleControllerManager
3:
4: moveit_simple_controller_manager:
5:   controller_names:
6:     - arm_controller
7:     - gripper_controller
8:
9:   arm_controller:
10:     action_ns: follow_joint_trajectory
11:     type: FollowJointTrajectory
12:     default: true
13:     joints:
14:       - joint_1
15:       - joint_2
16:       - joint_3
17:       - joint_4
18:       - joint_5
19:
20:   gripper_controller:
21:     action_ns: follow_joint_trajectory
22:     type: FollowJointTrajectory
23:     default: true
24:     joints:
25:       - joint_6
26:       - joint_7
27:
```

[File: pilz_cartesian_limits.yaml \(Configuration File\)](#)

```
1: cartesian_limits:
2:   max_trans_vel: 0.5   # Slower translational speed
3:   max_trans_acc: 1.0   # Reduced acceleration for smoother starts
4:   max_trans_dec: -2.5  # Softer deceleration for controlled stops
5:   max_rot_vel: 0.78    # Slower rotational speed (approximately 45 degrees per second)
6:
```

File: [planning_python_api.yaml](#) (Configuration File)

```
1: plan_request_params:
2:   planner_id: ompl
3:   planning_attempts: 2 # Increase for better solution robustness
4:   planning_pipeline: ompl
5:   max_velocity_scaling_factor: 0.75 # Slower speed for precise control
6:   max_acceleration_scaling_factor: 0.75 # Reduced acceleration for smoother movement
7:
```

src/armbot_moveit/launch

[File: moveit.launch.py \(Launch File\)](#)

```
1: import os
2: from launch import LaunchDescription
3: from moveit_configs_utils import MoveItConfigsBuilder
4: from launch_ros.actions import Node
5: from launch.actions import DeclareLaunchArgument
6: from launch.substitutions import LaunchConfiguration
7: from ament_index_python.packages import get_package_share_directory
8:
9:
10: def generate_launch_description():
11:     # Declare the argument for simulation mode
12:     is_sim = LaunchConfiguration('is_sim')
13:
14:     is_sim_arg = DeclareLaunchArgument(
15:         'is_sim',
16:         default_value='True',
17:         description='Flag to indicate if the robot is running in simulation mode'
18:     )
19:
20:     # Build MoveIt configurations for armbot
21:     moveit_config = (
22:         MoveItConfigsBuilder("armbot", package_name="armbot_moveit")
23:         .robot_description(file_path=os.path.join(
24:             get_package_share_directory("armbot_description"),
25:             "urdf",
26:             "armbot.urdf.xacro"
27:         ))
28:         .robot_description_semantic(file_path="config/armbot.srdf")
29:         .trajectory_execution(file_path="config/moveit_controllers.yaml")
30:         .to_moveit_configs()
31:     )
32:
33:     # Move group node
34:     move_group_node = Node(
35:         package="moveit_ros_move_group",
36:         executable="move_group",
37:         output="screen",
38:         parameters=[
39:             moveit_config.to_dict(),
40:             {'use_sim_time': is_sim},
41:             {'publish_robot_description_semantic': True}
42:         ],
43:         arguments=["--ros-args", "--log-level", "info"],
44:     )
45:
46:     # RViz node configuration
47:     rviz_config = os.path.join(
48:         get_package_share_directory("armbot_moveit"),
```

```

49:         "config",
50:         "moveit.rviz",
51:     )
52:     rviz_node = Node(
53:         package="rviz2",
54:         executable="rviz2",
55:         name="rviz2",
56:         output="log",
57:         arguments=["-d", rviz_config],
58:         parameters=[
59:             moveit_config.robot_description,
60:             moveit_config.robot_description_semantic,
61:             moveit_config.robot_description_kinematics,
62:             moveit_config.joint_limits,
63:         ],
64:     )
65:
66:     # Return the LaunchDescription
67:     return LaunchDescription([
68:         is_sim_arg,
69:         move_group_node,
70:         rviz_node
71:     ])
72:

```

src/armbot_msgs

File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.ros.org/schema/package_format3"?>
3: <package format="3">
4:   <name>armbot_msgs</name>
5:   <version>0.0.0</version>
6:   <description>Definition of Interfaces for the Armbot</description>
7:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
8:   <license>Apache 2.0</license>
9:
10:   <buildtool_depend>ament_cmake</buildtool_depend>
11:   <build_depend>roscpp</build_depend>
12:
13:   <depend>std_msgs</depend>
14:   <depend>action_msgs</depend>
15:
16:   <exec_depend>roscpp</exec_depend>
17:
18:   <member_of_group>roscpp</member_of_group>
19:
20:   <test_depend>ament_lint_auto</test_depend>
21:   <test_depend>ament_lint_common</test_depend>
22:
23:   <export>
24:     <build_type>ament_cmake</build_type>
25:   </export>
26: </package>
27:
```

src\armbot_msgs\action

File: ArmbotTask.action (Action Definition)

```
1: # Goal
2: int32 task_number
3: ---
4: # Result
5: bool success
6: ---
7: # Feedback
8: int32 percentage
```

File: Fibonacci.action (Action Definition)

```
1: # Goal
2: int32 order
3: ---
4: # Result
5: int32[] sequence
6: ---
7: # Feedback
8: int32[] partial_sequence
9:
```


src\armbot_msgs\srv

File: AddTwoInts.srv (Service Definition)

```
1: # Request
2: int64 a
3: int64 b
4: ---
5: # Response
6: int64 sum
```

File: EulerToQuaternion.srv (Service Definition)

```
1: # Request
2: float64 roll
3: float64 pitch
4: float64 yaw
5: ---
6: # Response
7: float64 x
8: float64 y
9: float64 z
10: float64 w
```

File: QuaternionToEuler.srv (Service Definition)

```
1: # Request
2: float64 x
3: float64 y
4: float64 z
5: float64 w
6: ---
7: # Response
8: float64 roll
9: float64 pitch
10: float64 yaw
```

src/armbot_utils

File: package.xml (Other)

```
1: <?xml version="1.0"?>
2: <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://w
3: <package format="3">
4:   <name>armbot_utils</name>
5:   <version>0.0.0</version>
6:   <description>The armbot_utils package</description>
7:   <maintainer email="jewelnath.me@gmail.com">Jewel Nath</maintainer>
8:   <license>Apache 2.0</license>
9:
10:   <buildtool_depend>ament_cmake</buildtool_depend>
11:   <buildtool_depend>ament_cmake_python</buildtool_depend>
12:
13:   <depend>rclcpp</depend>
14:   <depend>rclpy</depend>
15:   <depend>armbot_msgs</depend>
16:   <depend>tf2</depend>
17:
18:   <exec_depend>tf_transformations</exec_depend>
19:   <exec_depend>python3-transforms3d</exec_depend>
20:
21:   <test_depend>ament_lint_auto</test_depend>
22:   <test_depend>ament_lint_common</test_depend>
23:
24:   <export>
25:     <build_type>ament_cmake</build_type>
26:   </export>
27: </package>
28:
```

src\armbot_utils\armbot_utils

File: [__init__.py](#) (Other)

1:

File: angle_conversion.py (Other)

```
1: #!/usr/bin/env python3
2: import rclpy
3: from rclpy.node import Node
4: from armbot_msgs.srv import EulerToQuaternion, QuaternionToEuler
5: from tf_transformations import quaternion_from_euler, euler_from_quaternion
6:
7: class AnglesConverter(Node):
8:
9:     def __init__(self):
10:         super().__init__("angle_conversion_service_server")
11:         self.euler_to_quaternion_ = self.create_service(EulerToQuaternion, "euler_to_quaternion", self.euler_to_quaternion_callback)
12:         self.quaternion_to_euler_ = self.create_service(QuaternionToEuler, "quaternion_to_euler", self.quaternion_to_euler_callback)
13:         self.get_logger().info("Angles conversion services are ready")
14:
15:     def euler_to_quaternion_callback(self, req, res):
16:         self.get_logger().info("Requested to convert euler angles roll: %f, pitch: %f, yaw: %f" % (req.roll, req.pitch, req.yaw))
17:         # Calculate quaternion from euler
18:         (res.x, res.y, res.z, res.w) = quaternion_from_euler(req.roll, req.pitch, req.yaw)
19:         self.get_logger().info("Corresponding quaternion x: %f, y: %f, z: %f, w: %f" % (res.x, res.y, res.z, res.w))
20:         return res
21:
22:     def quaternion_to_euler_callback(self, req, res):
23:         self.get_logger().info("Requested to convert quaternion x: %f, y: %f, z: %f, w: %f" % (req.x, req.y, req.z, req.w))
24:         # Calculate euler from quaternion
25:         (res.roll, res.pitch, res.yaw) = euler_from_quaternion([req.x, req.y, req.z, req.w])
26:         self.get_logger().info("Corresponding euler angles roll: %f, pitch: %f, yaw: %f" % (res.roll, res.pitch, res.yaw))
27:         return res
28:
29:
30: def main():
31:     rclpy.init()
32:
33:     angles_converter = AnglesConverter()
34:     rclpy.spin(angles_converter)
35:
36:     angles_converter.destroy_node()
37:     rclpy.shutdown()
38:
39:
40: if __name__ == '__main__':
41:     main()
42:
```

src\armbot_utils\src

[File: angle_conversion.cpp \(Other\)](#)

```
1: #include <rclcpp/rclcpp.hpp>
2: #include "armbot_msgs/srv/euler_to_quaternion.hpp"
3: #include "armbot_msgs/srv/quaternion_to_euler.hpp"
4: #include <tf2/utils.h>
5:
6: #include <memory>
7:
8:
9: using namespace std::placeholders;
10:
11: class AnglesConverter : public rclcpp::Node
12: {
13: public:
14:     AnglesConverter() : Node("simple_service_server")
15:     {
16:         euler_to_quaternion_ = create_service<armbot_msgs::srv::EulerToQuaternion>("euler_to_quaternion",
17:             std::bind(&AnglesConverter::eulerToQuaternionCallback, this, _1, _2));
18:         quaternion_to_euler_ = create_service<armbot_msgs::srv::QuaternionToEuler>("quaternion_to_euler",
19:             std::bind(&AnglesConverter::quaternionToEulerCallback, this, _1, _2));
20:         RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Angles conversion services are ready");
21:     }
22:
23:
24: private:
25:     rclcpp::Service<armbot_msgs::srv::EulerToQuaternion>::SharedPtr euler_to_quaternion_;
26:     rclcpp::Service<armbot_msgs::srv::QuaternionToEuler>::SharedPtr quaternion_to_euler_;
27:
28:     void eulerToQuaternionCallback(const std::shared_ptr<armbot_msgs::srv::EulerToQuaternion> req,
29:                                     const std::shared_ptr<armbot_msgs::srv::EulerToQuaternion> res)
30:     {
31:         RCLCPP_INFO_STREAM(rclcpp::get_logger("rclcpp"), "Requested to convert euler angles: roll: " << req->roll,
32:                                     "pitch: " << req->pitch << ", yaw: " << req->yaw << ", into a quaternion.");
33:
34:         // Calculate quaternion from euler
35:         tf2::Quaternion q;
36:         q.setRPY(req->roll, req->pitch, req->yaw);
37:         res->x = q.getX();
38:         res->y = q.getY();
39:         res->z = q.getZ();
40:         res->w = q.getW();
41:         RCLCPP_INFO_STREAM(rclcpp::get_logger("rclcpp"), "Corresponding quaternion x: " << res->x,
42:                                     "y: " << res->y << ", z: " << res->z << ", w: " << res->w);
43:     }
44:
45:
46:     void quaternionToEulerCallback(const std::shared_ptr<armbot_msgs::srv::QuaternionToEuler> req,
47:                                     const std::shared_ptr<armbot_msgs::srv::QuaternionToEuler> res)
48:     {
```

```

49:         RCLCPP_INFO_STREAM(rclcpp::get_logger("rclcpp"), "Requested to convert quaternion x
50:                                     ", y: " << req->y << ", z: " << req->z << ",
51:                                     ", w: " << req->w << ", into euler angles: " << res->roll << ",
52:         // Calculate euler from quaternion
53:         tf2::Quaternion q(req->x, req->y, req->z, req->w);
54:         tf2::Matrix3x3 m(q);
55:         m.getRPY(res->roll, res->pitch, res->yaw);
56:         RCLCPP_INFO_STREAM(rclcpp::get_logger("rclcpp"), "Corresponding euler angles roll: " << res->roll << ",
57:                                     ", pitch: " << res->pitch << ", yaw: " << res->yaw << "\n");
58:     }
59:
60: };
61:
62:
63: int main(int argc, char* argv[])
64: {
65:     rclcpp::init(argc, argv);
66:     auto node = std::make_shared<AnglesConverter>();
67:     rclcpp::spin(node);
68:     rclcpp::shutdown();
69:     return 0;
70: }
71:

```